

ONIT-PNG

Observatoire National Intelligent
des Telecommunications

Republique de Guinee - ARPT

Audit Complet de la Plateforme

Phase 2 - Backend, Frontend, Securite et Production

Date : 17 mai 2026

Version : 2.0

Classification : Interne

Table des Matieres

1. Resume Executif	4
2. Architecture Backend	4
2.1 Stack Technique	4
2.2 Routes API	5
2.3 Points Forts du Backend	6
2.4 Problemes Identifies - Backend	6
3. Interface Frontend	7
3.1 Composants et Navigation	7
3.2 Points Forts du Frontend	7
3.3 Problemes Identifies - Frontend	8
4. Authentification et RBAC	8
4.1 Systeme d'Authentification	8
4.2 Modele RBAC	8
4.3 Problemes Identifies - Auth/RBAC	9
5. Base de Donnees	10
5.1 Schema Prisma	10
5.2 Problemes Critiques - Base de Donnees	10
6. Securite	10
6.1 Mesures de Securite Existantes	10

6.2 Vulnerabilites Identifiees	11
7. Deploiement et Production	11
7.1 Infrastructure Docker	11
7.2 Problemes de Production	11
8. Recommandations Prioritaires	12
8.1 Actions Critiques (Bloquantes pour la Production)	12
8.2 Actions Importantes (Avant Mise en Production)	12
8.3 Actions Souhaitees (Ameliorations Continues)	13
9. Conclusion	13

1. Resume Executif

Ce rapport presente l'audit complet de la plateforme ONIT-PNG (Observatoire National Intelligent des Telecommunications), developpee pour l'Autorite de Regulation des Postes et Telecommunications (ARPT) de la Republique de Guinee. L'audit couvre l'ensemble des couches techniques : architecture backend, interface frontend, systeme d'authentification, modele de donnees, securite, et prete a la production. La plateforme est construite sur Next.js 16 avec TypeScript, Prisma ORM, SQLite, et NextAuth v4, deployee via Docker avec un reverse proxy Caddy.

L'audit revele une plateforme fonctionnellement solide avec une couverture backend comprehensive (11 routes API), un systeme RBAC complet (9 roles, permissions granulaires, politiques RLS), et une interface utilisateur moderne comprenant 9 tableaux de bord specialises. Cependant, plusieurs vulnerabilites de securite et limitations techniques necessitent une attention prioritaire avant un deploiement en production, notamment le stockage de donnees sensibles dans les JWT, l'absence de validation des entrees sur certaines routes publiques, et la base SQLite non adaptee a la montee en charge.

Domaine	Score	Statut
Architecture Backend	8.5/10	BON
Interface Frontend	8/10	BON
Authentification & RBAC	7.5/10	MOYEN
Securite	6.5/10	ATTENTION
Base de Donnees	6/10	CRITIQUE
Production-Readiness	6.5/10	ATTENTION
Score Global	7.2/10	ATTENTION

Tableau 1 : Synthese des scores d'audit par domaine

2. Architecture Backend

2.1 Stack Technique

La plateforme est construite sur un stack moderne et coherant. Le framework Next.js 16 (React 19) est utilise en mode standalone pour une production optimisee, avec un output statique et un serveur Node.js leger. TypeScript est active avec ignoreBuildErrors=false, garantissant la verification de type a la compilation. Le runtime Node.js 20 Alpine est utilise dans le Dockerfile multi-stage, et Bun est disponible comme alternative de package manager. Le mode strict React est active, ce qui aide a detecter les bugs potentiels en developpement.

Composant	Technologie	Version
-----------	-------------	---------

Framework	Next.js	16.1.1
UI Framework	React	19.0.0
ORM	Prisma	6.11.1
Base de donnees	SQLite	N/A
Authentification	NextAuth	4
Validation	Zod	4.4.3
Hashing	bcryptjs	3.0.3
Cartographie	Leaflet / React-Leaflet	1.9.4 / 5.0.0
Graphiques	Recharts	2.15.4
UI Components	Radix UI + shadcn/ui	Multiple

Tableau 2 : Stack technique de la plateforme

2.2 Routes API

La plateforme expose 11 routes API couvrant l'ensemble des fonctionnalites metier. Chaque route implemente une verification d'authentification via `getSession` et un controle de permissions RBAC via `checkPermission`. Les routes de lecture publiques (dashboard, scoring, map, reports) acceptent les utilisateurs non authentifies avec un role PUBLIC par default, tandis que les routes d'ecriture requierent des permissions specifiques. La validation des entrees est systematiquement effectuee via Zod avec des schemas stricts incluant une sanitisation HTML (`stripHtml`). La pagination est implementee sur la route `/api/mesures` avec parametres `limit/offset`.

Route	Methode	Auth	RBAC	Zod
/api/dashboard	GET	Optionnel	RLS	Non
/api/campaigns	GET/POST	Requis	write	Oui
/api/mesures	GET/POST/ PUT	Requis	write	Oui
/api/qos	GET	Requis	RLS	Non
/api/scoring	GET	Optionnel	RLS	Non
/api/scores	GET/POST	GET: Optionnel	POST: write	Oui
/api/alerts	GET/POST/ PATCH	GET/POST: Optionnel	PATCH: write	Oui
/api/map	GET	Optionnel	RLS	Non
/api/reports	GET/POST	GET: Optionnel	POST: write	Oui
/api/users	GET/POST/ PATCH	Requis	admin	Oui

/api/roles	GET	Requis	admin	Non
/api/audit-logs	GET	Requis	read/admin	Non

Tableau 3 : Inventaire des routes API

2.3 Points Forts du Backend

Import en masse CSV/JSON : La route PUT /api/mesures supporte l'import en masse de mesures QoS dans deux formats (JSON et CSV) avec validation Zod sur chaque ligne, resolution automatique des codes operateur/region en IDs, et insertion par lots de 100 elements. Le CSV supporte des noms de colonnes flexibles (français et anglais) avec tolerance d'erreur par ligne.

Audit trail complet : Chaque action d'ecriture (CREATE, UPDATE, LOGIN) est enregistree dans la table AuditLog avec l'identifiant utilisateur, l'action, la ressource, les details serialises en JSON, et optionnellement l'adresse IP et le user-agent. Les connexions echouees sont egalement tracees pour la surveillance de securite.

Upsert des scores : La route POST /api/scores utilise Prisma upsert pour creer ou mettre a jour les scores operateur par periode, evitant les doublons grace a la contrainte unique operateurId + periode.

Sanitisation HTML systematique : Tous les schemas Zod pour les champs texte appliquent une fonction stripHtml qui supprime les balises HTML, prevenant les attaques XSS dans les donnees persistees.

2.4 Problemes Identifies - Backend

1. Absence de validation Zod sur les routes GET : Les routes /api/dashboard, /api/qos, /api/scoring, /api/map, /api/roles et /api/audit-logs n'appliquent pas de validation Zod sur les parametres de requete (query params). Bien que les parametres soient utilises dans les filtres Prisma, un utilisateur malveillant pourrait injecter des valeurs inattendues. Par exemple, le parametre 'limit' dans /api/mesures est correctement clamp (max 5000), mais les parametres de filtre comme 'operateur' ou 'region' sont directement utilises dans les requetes findFirst sans validation structuree.

2. Donnees simulees en dur : Plusieurs routes API utilisent des valeurs par default simulees lorsque les donnees reelles sont insuffisantes. Par exemple, /api/dashboard ajoute +200 aux zones blanches comptees (ligne : zonesBlanches + 200), /api/qos utilise des valeurs de disponibilite hardcodees (99 - 1.9 pour Celcom, etc.), et le score 'innovation' est calcule comme un simple pourcentage du score QoS. Ces artifices faussent les indicateurs presentes aux decideurs.

3. Requetes N+1 potentielles : Dans /api/mesures PUT (import CSV), chaque ligne resout independamment l'operateur et la region via des findFirst, generant potentiellement des milliers de requetes pour un import massif. Un cache ou une resolution par lot ameliorerait significativement les performances.

4. Gestion d'erreur insuffisante : Toutes les routes API catchent les erreurs de maniere generique avec console.error et retournent un message vague 'Erreur serveur'. Aucune differentiation entre erreurs de validation, erreurs de base de donnees, et erreurs d'authentification interne. Les logs console.error ne sont pas structures et ne facilitent pas le monitoring en production.

3. Interface Frontend

3.1 Composants et Navigation

L'interface utilisateur est organisee autour d'un layout principal (OnitLayout) avec une barre laterale de navigation, un en-tete avec recherche et notifications, et 9 tableaux de bord specialises. La navigation est filtrees par role : les onglets 'Cybersecurite' et 'Administration' ne sont visibles que pour les roles DIRECTEUR_TECHNIQUE+ et SUPER_ADMIN respectivement. L'interface est sombre (Dark Mode permanent) avec une palette elegante bleu-noir (#0A0F1E) et des accents dore (#D4A843) correspondant a l'identite visuelle institutionnelle guineenne.

Tableau de Bord	Composant	Fonctionnalites	Acces
Directeur	DashboardDG	KPIs, carte, classement operateurs, alertes	Tous auth
Monitoring QoS	DashboardQoS	Metriques, tendances, benchmark, heatmap	Tous auth
Cartographie SIG	DashboardSIG	Carte Leaflet, couches, points de mesure	Tous auth
Scoring	DashboardScoring	Radar, historique, recommandations IA	Tous auth
Audit Terrain	DashboardAudit	Campagnes, resultats drive test	Tous auth
Rapports	DashboardReports	Generation, telechargement, planification	Tous auth
Portail Public	DashboardPublic	KPIs, carte, signalement, FAQ	Public
Cybersecurite	DashboardCyber	Score securite, RBAC, chiffrement, audit	Dir. Tech+
Administration	DashboardAdmin	Users, roles, journal audit, systeme	Super Admin

Tableau 4 : Inventaire des tableaux de bord

3.2 Points Forts du Frontend

Cartographie Leaflet interactive : Le composant GuineaMapLeaflet affiche une carte interactive de la Guinee avec des marqueurs par operateur, des polygones de couverture regionaux, et des couches selectionnables (couverture, heatmap QoS, zones blanches, axes routiers). Le chargement dynamique est correctement gere avec un composant Inner charge cote client uniquement.

Notifications en temps reel : Le systeme de notifications interroge /api/alerts toutes les 30 secondes, affiche les alertes non resolues avec un badge compteur, et permet la resolution en masse. L'interface distingue les severites par couleur et icone.

Portail citoyen : Le DashboardPublic offre un acces non authentifie avec des KPIs de transparence, une carte de couverture, la comparaison des operateurs, un formulaire de signalement de probleme, et une

FAQ. Ce portail répond à l'obligation de transparence réglementaire de l'ARPT.

Administration complete : Le DashboardAdmin permet la gestion des utilisateurs (création, activation/désactivation), la visualisation des rôles et permissions, la consultation du journal d'audit, et le suivi de l'état du système. La protection contre l'auto-désactivation est implémentée (un admin ne peut pas désactiver son propre compte).

3.3 Problèmes Identifiés - Frontend

1. Téléchargement de rapports simulé : Le bouton de téléchargement dans DashboardReports génère un fichier texte/CSV basique avec des données en dur, et non un vrai rapport PDF avec les données réelles de la base. Le format PDF est déclaré mais le contenu est texte brut avec extension .txt. Ceci ne répond pas aux exigences réglementaires d'un rapport officiel.

2. Résultats d'audit en dur : Le DashboardAudit contient un tableau auditResults avec des valeurs codées en dur pour les résultats de test (latence, débit, taux d'appel). Ces données ne sont pas récupérées depuis l'API mais directement écrites dans le composant, ce qui signifie qu'elles ne reflètent jamais les données réelles.

3. Absence de gestion d'erreur utilisateur : Lorsqu'une requête API échoue, les composants affichent uniquement un message 'Chargement...' ou basculent sur des valeurs par défaut silencieusement. Aucune notification d'erreur n'est présentée à l'utilisateur pour les routes de lecture (dashboard, QoS, scoring, map). Seules les opérations d'écriture utilisent toast.error via Sonner.

4. Hydratation et performances : Le composant page.tsx utilise useSyncExternalStore pour la détection du montage client, ce qui est correct mais inhabituel. Le rechargement complet de la page après connexion (window.location.href = '/') est brut et pourrait être remplacé par un rafraîchissement de session plus élégant.

4. Authentification et RBAC

4.1 Système d'Authentification

L'authentification repose sur NextAuth v4 avec la stratégie JWT (maximum 8 heures de session). Le fournisseur CredentialsProvider utilise bcryptjs (cost factor 12) pour la vérification des mots de passe. Les cookies de session sont correctement configurés avec les attributs de sécurité : httpOnly, sameSite=lax, et secure en production HTTPS. Le middleware implémente un rate limiter en mémoire (5 tentatives par IP en 15 minutes) avec nettoyage automatique des entrées expirées. Les en-têtes de sécurité sont ajoutés à toutes les réponses : X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, Referrer-Policy et Permissions-Policy.

4.2 Modèle RBAC

Le modèle de contrôle d'accès basé sur les rôles (RBAC) est implémenté avec 9 rôles hiérarchisés allant de SUPER_ADMIN (accès complet) à PUBLIC (données publiques uniquement). Chaque rôle possède des permissions granulaires définies par paire ressource:action (ex: campaign:write, scoring:read, user:admin). Les politiques d'accès aux données (RLS - Row Level Security) sont définies dans la table

DataAccessPolicy avec trois scopes : 'all' (toutes les donnees), 'own_region' (donnees de ses regions uniquement), 'own_org' (donnees de son operateur), et 'public_only' (donnees publiques uniquement).

Role	Priorite	Scope RLS	Permissions
SUPER_ADMIN	100	all	24 permissions
DG	90	all	12 permissions
DGA	80	all	11 permissions
DIRECTEUR_TECHNIQUE	70	all	11 permissions
INGENIEUR_RF	60	own_region	8 permissions
ANALYSTE_QOS	50	all	9 permissions
AUDITEUR	40	all	8 permissions
OPERATEUR_READONLY	20	own_org	4 permissions
PUBLIC	10	public_only	2 permissions

Tableau 5 : Matrice des roles et permissions

4.3 Problemes Identifies - Auth/RBAC

1. Donnees sensibles dans les JWT : Le token JWT contient les permissions completes de l'utilisateur sous forme de tableau de chaines (resource:action). Avec le role SUPER_ADMIN et ses 24 permissions, le JWT peut devenir volumineux, dépassant potentiellement la limite de taille des cookies (4 Ko). De plus, les permissions sont figees dans le token a la connexion et ne sont pas rafraichies si un admin modifie les permissions d'un role pendant la session.

2. Pas de changement de mot de passe : L'interface utilisateur offre un bouton 'Changer le mot de passe' dans le menu utilisateur, mais celui-ci ne declenche aucune action. Il n'existe aucune route API pour le changement de mot de passe, ni de mecanisme de reinitialisation. Les mots de passe sont definis une seule fois a la creation et ne peuvent jamais etre modifies par l'utilisateur.

3. Comptes demo avec meme mot de passe : Le seed.ts definit le meme mot de passe 'Admin@2026!' pour les 10 comptes utilisateurs. Bien que ce soit acceptable pour un environnement de demonstration, c'est inacceptable en production. Aucun mecanisme ne force le changement de mot de passe a la premiere connexion.

4. Rate limiter en memoire : Le rate limiter du middleware est stocke en memoire dans un Map, ce qui signifie qu'il est reinitialise a chaque redemarrage du serveur et ne peut pas etre partage entre plusieurs instances. En cas de deploiement avec plusieurs conteneurs, le rate limiting sera inefficace.

5. Base de Donnees

5.1 Schema Prisma

Le modele de donnees Prisma comprend 10 modeles couvrant les domaines metier : Role, Permission, User, DataAccessPolicy, Operateur, Region, Campagne, MesureQoS, ScoreOperateur, Alerte, Rapport, AuditLog et Session. Les relations sont correctement definies avec des clees etrangeres et des contraintes d'unicite (operateurId + periode pour les scores, email pour les utilisateurs, ressource + action + roleId pour les permissions). Le modele MesureQoS est particulierement riche avec 22 metriques optionnelles couvrant la radio (RSSI, RSRP, RSRQ, SINR), le reseau (latence, debit, gigue, taux d'appel), Internet (download, upload, ping, DNS, TCP) et l'experience utilisateur (scoreQoE, pageLoadTime, videoBuffering).

5.2 Problemes Critiques - Base de Donnees

1. SQLite en production : C'est le probleme le plus critique pour le deploiement en production. SQLite est une base de donnees fichier mono-utilisateur qui ne supporte pas les connexions concurrentes en ecriture. En cas d'acces simultane de plusieurs utilisateurs, la base se verrouille et les ecritures echouent avec SQLITE_BUSY. De plus, SQLite ne supporte pas le chiffrement au repos (sauf extension SEE payante), ce qui pose un probleme de conformite pour les donnees personnelles (RGPD/SOA). La migration vers PostgreSQL est imperative pour la production.

2. Absence de migrations : Le projet utilise prisma db push au lieu de prisma migrate dev pour les changements de schema. Cette approche ne genere pas de fichiers de migration, rendant impossible le suivi des modifications de schema dans le temps, les retours en arriere, et la reproduction fiable de la base dans differents environnements. Le Dockerfile lui-meme execute prisma db push --skip-generate au demarrage, ce qui est risqué car cela peut modifier le schema de maniere inattendue en production.

3. Pas d'index explicites : Malgre les requetes frequentes filtrant par operateurId, regionId, campagneId et typeMesure dans MesureQoS, aucun index explicite n'est defini dans le schema Prisma. Les seuls indexes implicites sont ceux des clees primaires et des contraintes @unique. Avec des milliers de mesures, les performances de requetes se degraderont significativement sans indexes composes.

6. Securite

6.1 Mesures de Securite Existantes

La plateforme implemente plusieurs mesures de securite notables. Les en-tetes de securite HTTP sont configures dans le middleware (X-Content-Type-Options: nosniff, X-Frame-Options: DENY, X-XSS-Protection: 1; mode=block, Referrer-Policy: strict-origin-when-cross-origin, Permissions-Policy restrictif). Le rate limiting sur la route de connexion est present avec 5 tentatives maximales par IP en 15 minutes. La sanitisation HTML est systematique via Zod. Le hashing des mots de passe utilise bcryptjs avec un cost factor de 12. L'en-tete X-Powered-By est supprime dans next.config.ts pour eviter l'empreinte framework.

6.2 Vulnerabilites Identifiees

- 1. Route de signalement publique sans limitation :** La route POST /api/alerts accepte des signalements publics (type SIGNALEMENT_PUBLIC) sans authentication ni rate limiting specifique. Un attaquant pourrait spammer cette route pour inonder la base de donnees d'alertes fictives, saturant l'equipe de surveillance. Le message est limite a 500 caracteres, mais aucune limitation de debit n'est appliquee.
- 2. Absence de CSP (Content Security Policy) :** Aucun en-tete Content-Security-Policy n'est defini. Sans CSP, la plateforme est vulnerable aux attaques XSS par injection de scripts malveillants, malgre la sanitisation HTML cote serveur. Un attaquant sophistique pourrait contourner les filtres et injecter du JavaScript dans les pages.
- 3. CORS non configure :** Aucune configuration CORS n'est visible dans le middleware ou next.config.ts. Par default, Next.js API routes acceptent les requetes de toute origine. En production avec le reverse proxy Caddy, cela pourrait permettre a des sites tiers d'appeler les API directement.
- 4. NEXTAUTH_SECRET en .env.example :** Le fichier .env.example contient la valeur 'changez-cette-valeur-en-production' pour NEXTAUTH_SECRET. Bien que le code n'ait pas de valeur par default (ce qui est correct), le risque est qu'un operateur deploie la plateforme avec cette valeur par default du fichier .env.example sans la modifier.

7. Deploiement et Production

7.1 Infrastructure Docker

Le Dockerfile utilise un build multi-stage en trois etapes : deps (installation des dependances), builder (compilation et generation Prisma), et runner (execution en production). Le runner utilise node:20-alpine, cree un utilisateur non-root (nextjs:nodejs), et expose le port 3000. Un healthcheck est configure sur /api/auth/session avec un intervalle de 30 secondes. Le docker-compose.yml definit un service unique avec un volume persistant pour la base de donnees SQLite et une variable NEXTAUTH_SECRET obligatoire. Le reverse proxy Caddy est configure pour le port 81 avec transfert des en-tetes de proxy (X-Forwarded-For, X-Real-IP, X-Forwarded-Proto).

7.2 Problemes de Production

- 1. prisma db push au demarrage :** La commande CMD du Dockerfile execute 'prisma db push --skip-generate && node server.js'. L'utilisation de db push en production est risque car elle peut appliquer des modifications de schema de maniere destructive sans confirmation. En cas de regression de schema, les donnees existantes pourraient etre perdues. Il faudrait utiliser prisma migrate deploy pour appliquer uniquement les migrations validees.
- 2. Pas de strategy de sauvegarde :** Aucun mecanisme de sauvegarde automatique de la base SQLite n'est configure. Le volume Docker onit-db persiste les donnees entre les redemarrages, mais il n'y a pas de sauvegarde planifiee, de replication, ou de strategy de restauration. Une perte de volume signifie une perte totale de donnees.

3. Pas de monitoring : Aucune solution de monitoring n'est configuree. Il n'y a ni logs structures (les erreurs sont envoyees a console.error), ni metriques d'application (Prometheus/Grafana), ni alerting automatique. Le healthcheck Docker est le seul mecanisme de surveillance, mais il ne verifie que la disponibilite du serveur, pas la sante de la base de donnees ou les performances des requetes.

4. Pas de HTTPS dans le Dockerfile : Le Dockerfile et docker-compose.yml sont configures pour HTTP uniquement. La terminaison TLS est supposee etre geree par le reverse proxy Caddy, mais aucun certificat TLS n'est configure dans le Caddyfile fourni. En production, Caddy peut generer automatiquement des certificats Let's Encrypt, mais cela necessite une configuration DNS et une exposition directe a Internet.

5. Absence de tests automatises : Le repertoire tests/ contient des modeles de donnees (CSV, JSON) et un script de test API basique (test_api_endpoints.sh), mais il n'y a aucun test unitaire, test d'integration, ou test de bout en bout automatisé. Aucun framework de test (Jest, Vitest, Playwright Test, Cypress) n'est configure dans package.json. C'est un risque majeur pour la maintenance et l'evolution de la plateforme.

8. Recommandations Prioritaires

8.1 Actions Critiques (Bloquantes pour la Production)

Priorit e	Action	Impact	Effort
P0	Migrer SQLite vers PostgreSQL	Concurrence, performance, chiffrement	3-5 jours
P0	Ajouter Content-Security-Policy	Protection XSS, conformite securite	0.5 jour
P0	Rate limiter sur POST /api/alerts	Prevention spam signalements	0.5 jour
P0	Supprimer donnees en dur dans API	Fiabilite des indicateurs	2-3 jours

Tableau 6 : Actions critiques (P0)

8.2 Actions Importantes (Avant Mise en Production)

Priorit e	Action	Impact	Effort
P1	Migrer vers prisma migrate deploy	Reproductibilite des schemas	1 jour
P1	Ajouter indexes Prisma sur MesureQoS	Performance des requetes	0.5 jour
P1	Implementer changement mot de passe	Securite des comptes	1-2 jours
P1	Configurer CORS strict	Prevention attaques cross-origin	0.5 jour

P1	Ajouter logs structures (pino/winston)	Observabilite en production	1-2 jours
P1	Configurer sauvegardes automatiques	Resilience des donnees	1 jour

Tableau 7 : Actions importantes (P1)

8.3 Actions Souhaitees (Ameliorations Continues)

Priorit e	Action	Impact	Effort
P2	Ajouter tests unitaires et E2E	Qualite, maintenance, regression	5-10 jours
P2	Resolution par lot pour imports CSV	Performance import masse	1 jour
P2	Generation de rapports PDF reels	Conformite reglementaire	3-5 jours
P2	Monitoring Prometheus/Grafana	Observabilite avancee	2-3 jours
P2	Internationalisation (i18n)	Accessibilite multilingue	2-3 jours
P2	Cache Redis pour sessions/rate limiting	Scalabilite multi-instance	2 jours

Tableau 8 : Actions souhaitees (P2)

9. Conclusion

La plateforme ONIT-PNG constitue une base solide et fonctionnellement riche pour la supervision des telecommunications en Guinee. L'architecture technique est coherente, le modele RBAC est bien concu avec des politiques RLS granulaires, et l'interface utilisateur offre une experience professionnelle avec 9 tableaux de bord specialises. La couverture fonctionnelle est large, couvrant le monitoring QoS, la cartographie SIG, le scoring des operateurs, l'audit terrain, la gestion des rapports, le portail citoyen, la cybers securite et l'administration.

Cependant, la plateforme n'est pas encore prete pour un deploiement en production dans son etat actuel. Les points bloquants principaux sont : (1) l'utilisation de SQLite qui ne supporte pas la concurrence en ecriture, (2) l'absence de Content-Security-Policy, (3) les donnees simulees en dur qui faussent les indicateurs, et (4) le manque de tests automatises. Ces problemes doivent etre resolus en priorite avant toute mise en production. Les recommandations P0 representent environ 6-9 jours de travail, les P1 environ 5-7 jours, et les P2 environ 15-23 jours. Avec un investissement de 2 a 3 semaines supplementaires, la plateforme pourrait atteindre un niveau de maturite production-ready conforme aux exigences reglementaires de l'ARPT.